IMT Arena : notre réflexion de conception

26 octobre 2026 · Badr et trinôme, CI1 FISA ITR · version 2, après revue "niveau senior"

Le projet en une phrase

Nous concevons en Java une librairie d'objets qui planifie, simule et archive les tournois e-sport de l'association IMT Arena. La simulation est pilotée par le temps et le hasard, dans des threads séparés, et seul le rapport de fin de tournoi est sauvegardé en base H2 via JPA.

Ce qui est évalué avant tout, c'est **comment** c'est conçu, pas seulement que ça tourne :

Critère	Points
Réalisation	8
Échanges (entretien de 10 min, un membre différent à chaque séance)	6
Assiduité et ponctualité	2
Qualité du code, du dossier de conception et du rendu	4

Notre fil conducteur : chaque règle métier est garantie par l'objet lui-même, chaque choix (collection, pattern, synchronisation) répond à un problème précis du sujet.

Nos choix de modélisation

Nous utilisons l'héritage pour les objets de même nature, les interfaces pour les capacités partagées entre objets de nature différente, et l'encapsulation pour interdire tout état incohérent.

Héritage (même nature, "est un")

- `Personne` (abstraite) : id, nom, prénom, pseudo, genre, téléphone
  - `Joueur` : une seule équipe, statut `AMATEUR` ou `PRO` (enum), compteur de victoires depuis son arrivée
  - `Coach` : une seule équipe, total de victoires en carrière
  - `Arbitre` : jeux qualifiés, nombre de matchs arbitrés
- `Jeu` (abstrait) : nom, nombre de manches, méthode abstraite `nbJoueursRequis()`
  - `JeuSolo` : 1 joueur aligné, désigné parmi l'équipe
  - `JeuEquipe` : de 3 à 5 joueurs selon le jeu (valeur fixée à la création)

Ajouter un nouveau type de jeu = créer une nouvelle sous-classe de `Jeu`, sans modifier l'existant (principe ouvert/fermé, exigé par le sujet).

Interfaces (capacités communes, "sait faire")

- `Classable` : `getPoints()`, implémentée par `Equipe` (et réutilisable plus tard pour un classement de joueurs)
- `Qualifiable` : `estQualifiePour(Jeu)`, implémentée par **`Arbitre`** (qualifié pour arbitrer ce jeu) **et `Equipe`** (a un coach et le bon nombre de joueurs pour ce jeu). Deux objets de nature différente, une même capacité : c'est exactement le cas décrit par le sujet.
- `MatchListener` : être prévenu de la fin d'un match (voir patterns)

Encapsulation (règles garanties par les objets)

- Pas de setter public dangereux : `equipe.ajouterJoueur(j)` lève une exception si le joueur a déjà une équipe

- La relation joueur/équipe est gérée **des deux côtés dans une seule méthode** (`ajouterJoueur` met aussi à jour `joueur.equipe`), et le compteur de victoires est remis à zéro quand le joueur change d'équipe
- Le constructeur de `Match` refuse deux équipes identiques et un arbitre non qualifié
- Le passage amateur vers pro se fait dans `Joueur`, jamais depuis l'extérieur
- Les listes exposées sont en lecture seule (`Collections.unmodifiableList`)
- Le statut du tournoi (`PLANIFIE`, `EN_COURS`, `TERMINE`) ne change que par `demarrer()` et `terminer()`
- **`equals` et `hashCode` redéfinis** sur `Jeu` et `Equipe` (sur le nom, qui est unique), car ces objets vont dans des `HashSet` et des `HashMap`
- **Exceptions métier dédiées**, non vérifiées : `TournoiInvalideException`, `RegleMetierException`, plutôt que des `RuntimeException` génériques

Autres classes

- `Equipe` : nom, coach, joueurs, points (victoire = 3, pas de nul)
- `Match` : deux équipes, jeu, arbitre, vainqueur, état joué ou annulé
- `Tournoi` : nom, jeu, date, budget, 1 à 4 arènes, équipes, matchs en tous contre tous
- `Arena` : la façade d'entrée, **qui délègue** à des registres en mémoire (`RegistrePersonnes`, `RegistreEquipes`) et à un `PlanificateurTournois`, pour éviter une "classe Dieu" qui fait tout
- `RapportTournoi` : l'entité JPA persistée

Nombre de matchs d'un tournoi à n équipes (6 équipes donnent 15 matchs) :

$$N_{\text{matchs}} = \frac{n\,(n-1)}{2}$$

Collections choisies

Chaque besoin du sujet a sa structure, choisie pour l'opération qu'on y fait le plus souvent.

Besoin	Structure	Justification
Tournoi suivant à lancer	`PriorityQueue<Tournoi>`, `Tournoi implements Comparable` (par date)	`poll()` donne toujours le plus proche en O(log n)
Liste des tournois planifiés, le plus proche d'abord	copie triée de la file (`stream().sorted()`)	**parcourir une `PriorityQueue` ne donne pas un ordre trié** : seule la tête est garantie
Classement des équipes, sans doublon, points puis nom	`TreeSet<Equipe>` construit **à la demande** avec un `Comparator` (points décroissants, puis nom)	instantané trié et sans doublon, recalculé à chaque consultation
Accès direct à une équipe par son nom	`HashMap<String, Equipe>`	recherche en O(1)
Jeux pour lesquels un arbitre est qualifié	`HashSet<Jeu>`	pas de doublon, `contains` rapide ; pas d'`EnumSet` car les jeux doivent rester extensibles
Matchs d'un tournoi groupés par vainqueur	`Map<Equipe, List<Match>>` via `Collectors.groupingBy`	regroupement direct ; `TreeMap` si on veut l'ordre alphabétique
Tournois terminés, du plus gros au plus petit budget	`ArrayList` triée par un `Comparator` sur le budget	tri ponctuel à l'affichage
Joueurs d'une équipe, matchs d'un tournoi	`ArrayList`	accès par index, peu d'insertions au milieu
Listeners d'un match	`CopyOnWriteArrayList<MatchListener>`	parcourue depuis le thread de simulation sans risque de modification concurrente

Pourquoi un classement calculé à la demande : un `TreeSet` trie à l'insertion. Si on le garde en permanence alors que les points changent, il faut penser à retirer puis réinsérer chaque équipe, et un oubli corrompt le classement sans aucune erreur visible. Le reconstruire au moment de l'afficher supprime ce risque, pour un coût négligeable avec 4 à 6 équipes. Le comparateur départage par le nom, sinon deux équipes à égalité de points seraient vues comme un doublon et l'une disparaîtrait.

Design patterns retenus

Nous retenons quatre patterns, chacun répondant à une situation décrite dans le sujet ; aucun n'est ajouté "pour faire joli".

Pattern	Problème résolu	Mise en oeuvre
Strategy	Le vainqueur peut être désigné de plusieurs façons, et on veut en changer sans toucher à `Match`	interface `StrategieVainqueur` avec `TirageEquitable`, `FavoriseMieuxClasse`, `ToujoursMieuxClasse` (utile pour des tests déterministes)
Observer	À la fin d'un match ou d'un incident, plusieurs éléments doivent être prévenus sans que le match les connaisse	interface `MatchListener` (`onMatchTermine`, `onIncident`) ; le classement, le journal et l'affichage s'abonnent ; les listeners doivent être thread-safe car appelés depuis le thread de simulation
Factory	Créer un jeu à partir d'un simple nom sans répéter de code	`JeuFactory` avec un **registre** `Map<String, Supplier<Jeu>>` : on enregistre un nouveau jeu sans toucher au code de la Factory (pas de `switch`), donc l'ouvert/fermé est respecté
Builder	Construire un tournoi valide (arènes entre 1 et 4, au moins 4 équipes, arbitre qualifié...) sans constructeur à 8 paramètres	`new TournoiBuilder().nom(...).jeu(...).arenes(3).build()` ; `build()` vérifie tout et lève une `TournoiInvalideException` si invalide

Pattern écarté : **State** pour le statut du tournoi. Trois états avec des transitions simples, un `enum` suffit ; un State serait artificiel.

Simulation et synchronisation

Chaque tournoi lancé tourne dans son propre thread, géré par un `ExecutorService` de 3 threads, et renvoie un `Future<RapportTournoi>`.

Principe directeur : le confinement plutôt que les verrous partout. Le thread de simulation travaille sur ses propres données (les matchs du tournoi) et ne publie le résultat qu'à la fin, d'un seul coup. Moins de données partagées, donc moins de `synchronized` et moins de bugs difficiles à reproduire.

Déroulement d'une simulation

1. Le tournoi passe à `EN_COURS` et génère ses matchs en tous contre tous.
2. La durée totale suit la formule ci-dessous. Exemple : 15 matchs, 100 ms par match, 3 arènes donnent 500 ms.
3. Pour chaque match : un arbitre qualifié est attribué (à tour de rôle), puis un tirage décide d'un incident (10 %, match annulé sans points), sinon la `StrategieVainqueur` désigne le gagnant.
4. Les `MatchListener` sont prévenus.
5. Fin automatique : points des équipes, victoires des coachs, matchs arbitrés, promotions des joueurs, statut `TERMINE`, réussite si au moins 70 % des matchs joués, création et sauvegarde du rapport.

$$D = \frac{N_{\text{matchs}} \times t_{\text{match}}}{N_{\text{arènes}}}$$

Testabilité par injection : le `Random` (graine fixe dans les tests), la `StrategieVainqueur` et le temps par match sont **passés au constructeur** du simulateur, jamais créés à l'intérieur. C'est ce qui rend les résultats reproductibles et les tests rapides.

Garanties de sûreté

Contrainte du sujet	Notre solution
Max 3 tournois en cours, même si deux lancements arrivent en même temps	`Semaphore(3)` avec `tryAcquire()` (refus immédiat plutôt qu'attente) ; la place est **libérée dans un `finally`**, sinon un tournoi qui plante bloquerait une place pour toujours
Le main consulte l'état sans voir de données incohérentes	statut dans un `AtomicReference<Statut>` ; le rapport est un objet **immuable** publié une seule fois à la fin
Récupérer le rapport avec un délai maximal	`future.get(timeout, TimeUnit.SECONDS)`, avec gestion de `TimeoutException`
Interruption pendant la simulation	`InterruptedException` capturée, puis `Thread.currentThread().interrupt()` pour restaurer le flag, jamais avalée
Arrêt propre, aucun thread restant	`shutdown()` puis `awaitTermination()` à la fin du main, `shutdownNow()` si le délai expire
Mise à jour des points en fin de tournoi	faite en une seule section `synchronized` sur le registre des équipes ; le classement est ensuite calculé à la demande

Une même équipe ne peut pas être dans deux tournois en cours (règle de démarrage), ce qui limite fortement les accès concurrents aux mêmes objets.

Persistance

Seul le rapport de tournoi est persisté, dans une base H2 fichier (`jdbc:h2:./data/arena`) via JPA (Hibernate).

- Entité `RapportTournoi` (`@Entity`) : id généré, nom du tournoi, date, jeu, matchs prévus, matchs joués, équipe vainqueur, réussite ; constructeur sans argument exigé par JPA, en `protected`
- Créé automatiquement à la fin de chaque tournoi, puis sauvegardé par un `RapportRepository`
- Deux requêtes : `trouverTous()` et `trouverParJeu(String jeu)` (JPQL)
- Configuration dans `META-INF/persistence.xml`

Les autres objets (équipes, joueurs, tournois) restent en mémoire : le sujet ne demande pas de les persister, et les garder hors de JPA simplifie le modèle.

Revue "niveau senior" : ce que nous avons corrigé

Une relecture critique de la première version a fait ressortir ces points, tous intégrés ci-dessus :

Problème de la v1	Correction en v2
`TreeSet` de classement gardé en permanence, fragile quand les points changent	classement reconstruit à la demande
Liste des tournois planifiés obtenue en parcourant la `PriorityQueue` (ordre non garanti)	file pour `poll()`, copie triée pour l'affichage
`equals` / `hashCode` absents sur des objets mis en `HashSet` / `HashMap`	redéfinis sur le nom
Place de la limite des 3 tournois jamais libérée en cas d'erreur	`tryAcquire()` et `release()` dans un `finally`
`synchronized` dispersés sur les objets métier	confinement, `AtomicReference`, rapport immuable, `CopyOnWriteArrayList`
Factory avec un `switch` (à modifier pour chaque jeu)	registre `Map<String, Supplier<Jeu>>`
`Qualifiable` utilisée par une seule classe	aussi implémentée par `Equipe`
`Random` et stratégie créés en interne, difficiles à tester	injectés par le constructeur
`Arena` risquait de tout faire	façade qui délègue à des registres et au planificateur

Plan d'action et répartition

Nous faisons d'abord fonctionner le coeur (modèle, tournoi, simulation), puis nous ajoutons persistance et tests, comme le conseille le sujet.

Étapes

- ☐ Étape 0 : diagramme de classes sur papier, projet Maven (JUnit 5, Hibernate, H2), repo Git partagé
- ☐ Étape 1 : personnes, jeux, équipes, avec les règles dans les objets et `equals` / `hashCode`
- ☐ Étape 2 : matchs, tournois, Builder, conditions de démarrage, classement
- ☐ Étape 3 : simulation multithread, Strategy, Observer, fin automatique
- ☐ Étape 4 : persistance JPA du rapport
- ☐ Étape 5 : au moins 3 tests JUnit variés (paramétré, `assertThrows`, `@BeforeEach`) et main de démonstration
- ☐ Étape 6 : dossier de 2 pages, README, ZIP testé depuis zéro

Répartition dans le trinôme

Membre	Responsabilité
A	Modèle (étapes 1 et 2), collections, Factory et Builder
B	Simulation et concurrence (étape 3), Strategy et Observer
C	JPA, tests, main de démo, dossier et README

L'entretien (6 points) se fait avec un membre différent à chaque séance : chacun doit pouvoir expliquer tout le projet. Avant chaque séance, 15 minutes où chacun présente sa partie aux deux autres.

Pièges à éviter

- `System.out.println` en dehors du main et des tests
- Patterns plaqués sans justification
- Match d'une équipe contre elle-même, joueur dans deux équipes, tournoi sans arbitre

- Threads encore actifs à la fin du programme
- Rendu autre qu'un ZIP ou en retard (note divisée par 2)

> **Note.** Cette réflexion de conception a été construite avec l'aide de Claude (assistant IA d'Anthropic) : résumé du sujet, proposition d'architecture, choix des collections et des patterns, puis relecture critique "niveau senior" ayant conduit aux corrections de cette version 2. Les choix ont été relus, compris et validés par le trinôme.